

Evolutionary Test Generation with Diversity-Aware Mutation Fitness

Minwook Lee, Minwoo Kim, Junho Hwang, and Seonwoo Ji CS453 Team 6
KAIST

Abstract—EvoSuite optimises mutation score by counting killed mutants, without considering which mutation operators they belong to. Two test suites can therefore achieve the same score while covering very different fault models. We extended EvoSuite’s strong-mutation fitness with Shannon entropy over killed-mutant operator types and coupled this diversity signal into the base objective through several modes (additive, multiplicative, exponential, and capped-multiplicative). A Python automation pipeline ran approximately 3,790 EvoSuite experiments across 14 Apache Commons and Guava classes in four phases. After fixing a population-size issue that blocked evolution in early phases, tuned diversity coupling improved median mutation score on 12 of 13 evaluated classes and jointly improved mutation score and operator entropy on 9 of 13. Effects are class-specific; `CharUtils` was the sole regression. Overall, the results partially support our hypothesis that diversity-aware selection can broaden fault-model exploration, but a single global setting does not work everywhere. Code: github.com/minux-lee/CS453-team6.

I. INTRODUCTION

Automated unit test generation is a practical way to improve software quality. EvoSuite [1], [2] uses an evolutionary algorithm to generate JUnit tests that maximise coverage and mutation-based criteria. The strong-mutation objective minimises branch distance plus per-mutant infection/propagation distances [2]. Once a mutant is killed, the operator type does not affect fitness further.

This creates a blind spot: two suites with the same mutation score can kill very different mixes of operator types. For example, one suite might kill mostly arithmetic mutants while another spreads kills across arithmetic, comparison, and condition-negation mutants. The second suite covers a broader fault model but receives identical selection pressure.

Our hypothesis is that coupling operator-type diversity into fitness steers search toward broader exploration and improves mutation score without sacrificing it. This report describes the problem, our EvoSuite extension, experiments, and a before/after comparison against a baseline with diversity coupling disabled.

II. PROBLEM AND HYPOTHESIS

Mutation testing measures how many artificial faults a suite detects [3]. EvoSuite’s strong-mutation fitness combines branch coverage with distances to alive mutants (lower is better).

The score counts *how many* mutants are killed, not *which* operator types they represent. EvoSuite uses nine primary operators (field deletion, statement deletion, unary insertion, variable replacement, constant replacement, arithmetic replace-

ment, comparison replacement, bitwise replacement, condition negation) plus an “other” bucket.

Hypothesis: a normalised Shannon-entropy measure over killed-mutant operator types, when coupled into fitness, improves mutation score and operator diversity together compared with the stock objective under the same time and population budget.

We use operator entropy as a proxy for fault-model breadth and do not validate against real faults (e.g., Defects4J).

III. APPROACH

A. Diversity Metric

Let c_i be killed mutants in operator bucket i , $p_i = c_i / \sum_j c_j$, and n the number of buckets present:

$$v = - \sum_{i=1}^n p_i \log p_i, \quad (1)$$

$$\hat{v} = v / \log n \in [0, 1], \quad \delta = 1 - \hat{v}. \quad (2)$$

$\hat{v}$ is normalised entropy; δ is *diversity deficiency* (0 = diverse, 1 = single-type concentration). Entropy is computed over killed mutants on each fitness evaluation.

B. Coupled Fitness

EvoSuite minimises fitness. The base value is $F_{\text{base}} = f_{\text{branch}} + \sum_{m \in \text{alive}} d_{\text{min}}(m)$. Table I lists coupling modes controlled by `-Ddiversity_coupling`, `-Ddiversity_k`, and `-Ddiversity_cap`.

TABLE I
COUPLED FITNESS MODES ($\delta = 1 - \hat{v}$).

Mode	Formula
NONE	F_{base}
MULT	$F_{\text{base}}(1 + k\delta)$
ADD	$F_{\text{base}} + k\delta$
EXP	$F_{\text{base}} e^{k\delta}$
CAP_MULT	$F_{\text{base}}(1 + \min(k\delta, c))$

Plain multiplicative coupling at large k can collapse mutation score by over-penalising low diversity; `CAPPED_MULTIPLICATIVE` limits the multiplier with cap c .

IV. IMPLEMENTATION

We started from the latest EvoSuite release on GitHub, integrated our changes into the project repository, and built with Maven. Table II lists the main modified files.

TABLE II
EVOsuite SOURCE CHANGES.

File	Change
StrongMutationSuiteFitness.java Properties.java	Operator bucketing, entropy, coupled fitness, stats export DiversityCoupling enum; diversity_k, diversity_cap
RuntimeVariable.java	MutantTypeEntropy, MutantTypeEntropyNorm

StrongMutationSuiteFitness.java. We restructured the evaluator to separate base mutation distances from diversity coupling, map mutants to operator buckets, and export entropy to `statistics.csv`. Seven defects in the initial skeleton (null-pointer risks, NaN propagation, inverted optimisation direction) were fixed during integration.

The repository also contains a Python automation/package (benchmark registry, parallel EvoSuite runner, resumable sweeps, phase drivers) and an analysis/package (CSV parsing, delta metrics, plotting).

V. EXPERIMENTAL SETUP

A. Benchmarks

We used 14 utility classes from `commons-math3`, `commons-lang3`, `commons-text`, `commons-codec`, and `Guava`. Table III lists the benchmarks and domains.

TABLE III
BENCHMARK CLASSES (14 TOTAL).

Class	Library	Dom.
ArithmeticUtils, Fraction, Precision, Complex	math3	num.
IntMath	Guava	num.
IEEE754rUtils, NumberUtils, CharUtils, BooleanUtils	lang3	mixed
WordUtils, LevenshteinDist.	text	str.
Strings	Guava	str.
Hex, Soundex	codec	enc.

B. Phases and Budgets

Experiments proceeded in four phases (Table IV). Phase A used a coarse k grid $\{0.25, 0.5, 1.0, 2.0, 4.0, 8.0\}$, population 50, 40s budget, and 3 seeds. Phase B re-ran promising configurations with 10 seeds. Phases C and D are the primary evidence: population 5, denser k grids, and 8 or 6 seeds.

TABLE IV
EXPERIMENTAL PHASES.

Ph.	#Cls	Pop.	Budget
A	14	50	40 s
B	14	50	40 s
C	8	5	60 s
D	6	5	120 s

Population artefact. With population 50 and short budgets, most Phase A/B runs reported `Generations=0`: the initial

random population exhausted the time limit before evolution occurred. Reducing population to 5 in Phases C/D enabled real search (~ 35 generations on average); all comparative claims below refer to Phase C/D.

Control settings. All runs used `STRONGMUTATION`, `-Dtest_archive=false`, and `NONE` as baseline. Metrics: mutation score (Coverage), normalised entropy (`MutantTypeEntropyNorm`), and deltas vs. matched baseline.

VI. RESULTS

A. Phase A/B Screening

Phase A/B established directional trends: `EXPONENTIAL` coupling failed to train reliably and was excluded from later phases. `ADDITIVE` and `MULTIPLICATIVE` with $k \in [0, 1.5]$ showed stable gains; plain `MULTIPLICATIVE` with $k > 2$ frequently collapsed mutation score, motivating `CAPPED_MULTIPLICATIVE`.

B. Phase C (Smaller Classes)

Phase C evaluated eight classes (`Soundex`, `Hex`, `Fraction`, `Precision`, `NumberUtils`, `IEEE754rUtils`, `CharUtils`, `Strings`) with a dense k grid, eight seeds, and population 5. Table V lists the best k per coupling mode selected by average mutation score.

TABLE V
PHASE C: BEST k PER COUPLING MODE.

Coupling	Best k
ADDITIVE	0.5
MULTIPLICATIVE	0.2
CAPPED_MULTIPLICATIVE	8.0 (cap 1.0)

`CharUtils` was an outlier: the baseline already achieved high mutation score and entropy; diversity pressure shifted selection toward operator balance at the cost of fewer killed mutants ($\Delta MS \approx -0.054$).

C. Phase D (Larger Classes)

Phase D targeted six larger classes (`ArithmeticUtils`, `Complex`, `BooleanUtils`, `WordUtils`, `Fraction`, `IntMath`) with a 120s budget. Table VI lists the best k values.

TABLE VI
PHASE D: BEST k PER COUPLING MODE.

Coupling	Best k
ADDITIVE	0.5
MULTIPLICATIVE	0.75
CAPPED_MULTIPLICATIVE	2.0 (cap 1.0)

D. Combined Phase C+D Summary

Across 13 unique classes covered by Phases C and D (8 + 6 minus overlapping `Fraction`): **12/13** improved median mutation score under the best tuned configuration per class; **9/13** improved both mutation score and normalised entropy at that configuration; **1/13** (`CharUtils`) regressed.

Table VII highlights notable per-class gains at best tuned settings.

TABLE VII
SELECTED PER-CLASS IMPROVEMENTS (PHASE C+D).

Class	Coupling	k	ΔMS
<code>Fraction</code>	MULT	0.75	+0.091
<code>Precision</code>	ADD	1.5	+0.079
<code>Soundex</code>	MULT	0.5	+0.044
<code>ArithmeticUtils</code>	CAP_MULT	0.5	+0.051
<code>CharUtils</code>	—	—	−0.054

Domain-level aggregation (Table VIII) shows mutation score improvements in numeric, boolean, and encoding domains; the string domain regressed only on `CharUtils`.

TABLE VIII
DOMAIN-LEVEL TREND AT BEST CONFIGURATION (PHASE C+D).

Domain	Trend
Numeric	Improved on most classes
Boolean	Improved
Encoding	Improved (<code>Soundex</code> , <code>Hex</code>)
String	Improved except <code>CharUtils</code>

With ≤ 8 seeds per setting, Mann–Whitney U tests ($p < 0.05$) rarely reached significance; we emphasise median effect sizes.

E. Before/After Interpretation

Relative to the NONE baseline under identical budgets:

- 1) Modest k (0.2–0.75) most often *increased* mutation score rather than trading it for entropy alone.
- 2) MULTIPLICATIVE with $k \geq 2$ reliably *degraded* mutation score.
- 3) CAPPED_MULTIPLICATIVE stabilised high- k behaviour (e.g., `ArithmeticUtils`).
- 4) Effects are *conditional*: no single (mode, k) dominated all classes.

Table IX summarises practical configuration choices from our experiments.

TABLE IX
PRACTICAL CONFIGURATION RECOMMENDATIONS.

Use case	Mode	k
Safe default	ADDITIVE	0.5–1.0
Strongest gains	MULTIPLICATIVE	0.5–0.75
High k without collapse	CAP_MULT	2–4 (cap 1.0)
Avoid	MULTIPLICATIVE	≥ 2

VII. LIMITATIONS AND FUTURE WORK

Phases C/D used population 5 and 60–120s budgets; results may differ with EvoSuite defaults and longer runs. `test_archive=false` isolates suite-fitness effects but lowers absolute scores. Operator entropy is a proxy, not validated against real faults [4]. Future work: larger budgets, per-generation tracking, phased objectives, and Defects4J evaluation.

VIII. CONCLUSION

We extended EvoSuite strong-mutation fitness with operator-type entropy coupling. On 13 benchmark classes, tuned settings improved mutation score on 12/13 and jointly improved score and entropy on 9/13. The hypothesis is partially supported: diversity-aware fitness helps on many classes but needs per-class tuning. A practical default is ADDITIVE with $k \approx 0.5$ –1.0; use CAPPED_MULTIPLICATIVE when stronger diversity pressure is needed.

REFERENCES

- [1] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for Java classes,” in *Proc. ISSTA*, 2011.
- [2] G. Fraser and A. Arcuri, “Achieving higher test quality with mutation-based test generation,” *IEEE Trans. Softw. Eng.*, vol. 40, no. 9, pp. 1041–1059, 2014.
- [3] M. Papadakis *et al.*, “Mutation testing advances,” *Adv. Comput.*, vol. 112, pp. 1–75, 2019.
- [4] R. Just *et al.*, “Are mutants a valid substitute for real faults?” in *Proc. FSE*, 2014.